

Hardware

Chassis: Serpent SRX8

Main computer: AGX Orin

LiDAR: Livox MID-360

Camera: ZED2

Motor: REDS V8

VESC: VESC 6MK VI HP

IMU (built in VESC)

Servo: HV Digital Servo

Battery: LIHV 2S*2

Software

The system is an integrated perception, planning, and control software stack for 1/10-scale autonomous head-to-head racing. It consists of LiDAR-based localization, LiDAR and camera-based opponent perception, opponent tracking, TEB-based generation of the drivable region, and MPPI-based control. All modules share a common representation: a Frenet coordinate frame defined relative to the track centerline. This lets the system handle the vehicle state and obstacle positions consistently, including on curved sections of track.

1. Localization

The localization module estimates the vehicle's global position and orientation through LiDAR scan matching and sensor fusion. It computes the global pose by registering the current scan against a prebuilt LiDAR map, then fuses this pose with IMU and wheel odometry in an EKF. The LiDAR-based pose is used as the reference for correcting accumulated drift, while the IMU and wheel odometry are used for high-frequency state prediction. The module outputs estimates of the vehicle's position, orientation, and velocity, and converts them into a driving state in the Frenet frame.

2. LiDAR Perception

The LiDAR perception module detects opponents and obstacles in the point cloud. It first

removes irrelevant points from the input cloud through ground removal, voxel downsampling, and region-of-interest filtering. Euclidean clustering then groups nearby points into discrete objects, and the module estimates each cluster's centroid and size. Each detected object is projected onto the global path and converted into obstacle information in the Frenet frame.

3. Camera Perception

The camera perception module detects opponent vehicles using a ZED2 stereo camera and a YOLO model. YOLO produces bounding-box detections of opponent vehicles in the image, and the ZED2 depth data is used to recover each detection's 3D position. The recovered position is transformed from the camera frame to the vehicle frame and then to the global frame, and finally expressed in Frenet coordinates. The camera thus complements LiDAR by providing semantic object information that LiDAR alone cannot easily supply.

4. Opponent Tracking

The opponent tracking module associates obstacle detections from the LiDAR and camera over time to estimate the opponent's position and velocity. An EKF-based tracker reduces detection noise and maintains a stable track even when detections drop out briefly. It also distinguishes static obstacles from moving opponents based on position change and velocity over a fixed time window. The planning module uses these tracked obstacles as constraints to determine the avoidance direction and the drivable region.

5. Local Planning

The local planning module uses the TEB Local Planner to generate a local trajectory for obstacle avoidance. Starting from the global path, TEB optimizes a feasible trajectory that maintains clearance from obstacles, subject to the vehicle's kinematic constraints and its velocity and acceleration limits. It also compares candidate paths that pass an obstacle on the left or on the right and selects the appropriate avoidance direction.

The trajectory produced by TEB is converted into a corridor, a drivable region the MPPI controller can use. Obstacle positions define the infeasible region, and the selected avoidance direction sets the left and right bounds of the corridor. TEB therefore decides the avoidance direction, and the corridor carries that decision to the controller as a safety boundary.

6. Control

The control module computes the final steering and speed commands using MPPI-based Model Predictive Control. From the current vehicle state and the corridor, MPPI samples multiple candidate control sequences over the planning horizon and rolls out each sequence through the vehicle model. It assigns a cost to each rollout based on progress along the path, control-input variation, and corridor violation. The control sequence is then updated as a weighted average of the sampled sequences, with lower-cost rollouts receiving higher weight. The controller applies the first input of the updated sequence.

Rollouts that leave the corridor receive a high cost, so they contribute little to the weighted update and have negligible influence on the applied command. In this way, the corridor from TEB acts as a soft safety constraint at the control stage. If the corridor is not updated for longer than a set interval, the controller falls back to a conservative centerline-following mode to avoid issuing unsafe commands.